

[ИНТЕГРАЦИЯ С ПРИЛОЖЕНИЯМИ]

Используйте более дешевый инференс в уже знакомых вам приложениях

Cheaper Inference поддерживает OpenAI-совместимые функции завершения диалога, API ответов без сохранения состояния для Codex и API сообщений, совместимый с Anthropic, для Claude Code. Большинству агентов, инструментов для написания кода и чат-приложений нужны только базовый URL, ключ API и идентификатор модели.

[ПОДКЛЮЧЕНИЕ К РАБОЧЕЙ ОБЛАСТИ]

Не закрывайте это руководство, пока не получите ключ

Откройте раздел «Ключи API» в новой вкладке, скопируйте одноразовый пароль и вернитесь сюда, чтобы завершить настройку.

Открытые ключи API

Используйте этот базовый URL-адрес

ИСПОЛЬЗУЙТЕ ЭТУ БАЗОВУЮ КОНФИГУРАЦИЮ URL

Копировать

`https://api.cheaperinference.com/v1`

Прежде чем начать

[Открыть ключи API](#), затем скопируйте точный идентификатор модели из [каталога действующих моделей](#). Храните ключи в переменной среды, если приложение это поддерживает.

Codex

Coding agent

Claude Code

Coding agent

OpenClaw

Agent platform

Hermes Agent

Agent CLI

OpenCode

Coding agent

OpenWork

Desktop agent

Open WebUI

Chat interface

Cline

VS Code agent

Cursor

AI code editor

Aider

Terminal pair programmer

LibreChat

Multi-user chat

Continue

IDE assistant

Интерфейс командной строки и настольное приложение Codex

Отправьте новые локальные задачи Codex через API ответов Cheaper Inference.

[Официальная ссылка на конфигурацию](#)

Поддерживается: потоковая передача ответов, вызовы функций, локальные инструменты Codex, такие как `apply_patch`, и учет использования. Эта конфигурация применяется к новым локальным задачам в интерфейсе командной строки Codex, настольном приложении и расширении для IDE. Она не затрагивает облачные чаты Codex.

1 Установите или проверьте Codex

Если `codex --version` уже выводит версию, переходите к шагу 2.

ТЕРМИНАЛ

Копировать

```
npm install --global @openai/codex
codex --version
```

2 Сделайте свой API-ключ доступным

Создайте ключ в [рабочей области API Keys](#). Замените заполнитель ниже, а затем выполните команду в том же окне терминала, которое вы будете использовать для запуска Codex. Это позволит задать ключ только для текущего сеанса терминала.

MACOS ИЛИ LINUX

Копировать

```
export CHEAPER_INFERENCE_API_KEY="ir_live_YOUR_API_KEY"
```

WINDOWS POWERSHELL

Копировать

```
$env:CHEAPER_INFERENCE_API_KEY = "ir_live_YOUR_API_KEY"
```

3 Добавьте пользовательский провайдер

Откройте `~/.codex/config.toml` и добавьте приведенную ниже конфигурацию. В настольном приложении выберите «Настройки» → «Конфигурация» → «Открыть config.toml». Замените `gpt-5.4` на любой доступный в настоящее время идентификатор текстовой модели из [актуального каталога](#); модели с надежным инструментарием лучше всего подходят для задач кодирования.

```
model = "gpt-5.4"
model_provider = "cheaper-inference"

[model_providers.cheaper-inference]
name = "Cheaper Inference"
base_url = "https://api.cheaperinference.com/v1"
env_key = "CHEAPER_INFERENCE_API_KEY"
wire_api = "responses"
```

► Используете настольное приложение macOS из Dock?

4 Перезапустите и проверьте

Codex CLI

В терминале, где вы установили ключ, откройте свой проект и выполните команду:

ИНСТРУМЕНТ ТОЛЬКО ДЛЯ ЧТЕНИЯ

Копировать

```
codex exec --model gpt-5.4 --sandbox read-only "Запустите pwd без изменени
```

Настольное приложение Codex

1. Полностью закройте и снова откройте приложение.
2. Запустите **новую локальную задачу**; существующая задача сохранит свою исходную модель.
3. Запрос: «проверить этот проект без изменения файлов и сообщить его фреймворк.»

Убедитесь, что все работает: откройте **Историю** Cheaper Inference. В выполненном запросе должны быть указаны выбранная модель, конечная точка `/v1/responses`, использование токена и списанная сумма.

► Устранение неполадок

► Ограничения API ответов

Claude Code

[Официальное руководство по шлюзу LLM](#)

Выполняйте локальные задачи Claude Code через совместимый с Anthropic Messages API.

Доступно всем пользователям: каждая учетная запись Cheaper Inference может быть подключена к Claude Code с помощью активного API-ключа с областью действия **Inference**. Не требуется учетная запись Anthropic или отдельная активация. По-прежнему действуют обычные ограничения на баланс кошелька, доступ к моделям и API-ключу.

Поддерживается: потоковая передача текста, локальные инструменты Claude Code, результаты работы инструментов, многоходовые задачи, визуальное восприятие, подсчет токенов и учет использования кошелька. Это влияет на трафик локального API Claude Code, но не на работу чатов claude.ai.

Область совместимости: Claude Code использует `POST /v1/messages` и `POST /v1/messages/count_tokens`. Это не полный API платформы Anthropic; см. ограничения совместимости ниже.

Установить или проверить Claude Code:

КОНФИГУРАЦИЯ CLAUDE CODE

Копировать

```
npm install --global @anthropic-ai/claude-code
claude --version
```

Создайте ключ в [рабочей области API Keys](#), затем экспортируйте настройки подключения в терминал, который запустит Claude Code. Claude Code добавляет `/v1/messages` сам, поэтому этот базовый URL намеренно не заканчивается на `/v1`.

КОНФИГУРАЦИЯ CLAUDE CODE

Копировать

```
экспортировать ANTHROPIC_BASE_URL="https://api.cheaperinference.com"
экспортировать ANTHROPIC_AUTH_TOKEN="ir_live_YOUR_API_KEY"
экспортировать ANTHROPIC_MODEL="claude-opus-4.8"
экспортировать ANTHROPIC_SMALL_FAST_MODEL="claude-opus-4.8"
```

```
ккод-модель Клода-opus-4.8
```

Windows PowerShell:

КОНФИГУРАЦИЯ CLAUDE CODE

Копировать

```
$env:ANTHROPIC_BASE_URL = "https://api.cheaperinference.com"
$env:ANTHROPIC_AUTH_TOKEN = "ir_live_YOUR_API_KEY"
$env:ANTHROPIC_MODEL = "claude-opus-4.8"
$env:ANTHROPIC_SMALL_FAST_MODEL = "claude-opus-4.8"

claude --model claude-opus-4.8
```

Проверить подключение

Сначала проверьте конечную точку Messages независимо от Claude Code. Успешный ответ подтверждает наличие ключа, идентификатора модели, формы запроса Anthropic, маршрутизации и расчетов с кошельком.

КОНФИГУРАЦИЯ CLAUDE CODE

Копировать

```
curl https://api.cheaperinference.com/v1/messages \
-H "X-API-Key: ir_live_YOUR_API_KEY" \
-H "Anthropic-Version: 2023-06-01" \
-H "Content-Type: application/json" \
-d '{
  "model": "claude-opus-4.8",
  "max_tokens": 64,
  "messages": [{"role": "user", "content": "Reply with: connected"}]
}'
```

Then run one non-interactive Claude Code task from the configured terminal:

CLAUDE CODE CONFIGURATION

Copy

```
claude -p "Reply with: Claude Code connected" --model claude-opus-4.8
```

Expected result: Claude Code prints a response, and History shows a settled `/v1/messages` request with the selected model and token usage.

Model selection: replace both model values with an exact text-model ID from the live catalog. Setting the small/fast model explicitly prevents background helper calls from selecting an Anthropic model ID that is not in your workspace catalog.

Authentication: `ANTHROPIC_AUTH_TOKEN` sends the key as a bearer token. The Messages endpoint also accepts `X-API-Key` for Anthropic SDK clients. Do not put a live key in a repository or shared settings file.

► Messages API compatibility boundaries

OpenClaw

[Official provider docs](#)

Add Cheaper Inference as a custom Chat Completions provider.

1. Set `CHEAPER_INFERENCE_API_KEY` in the environment that starts OpenClaw.
2. Add the provider and model to your OpenClaw configuration.
3. Select the model as `cheaper-inference/gpt-5.4`.

OPENCLAW CONFIGURATION

Copy

```
{
  "env": {
    "CHEAPER_INFERENCE_API_KEY": "ir_live_YOUR_API_KEY"
  },
  "agents": {
    "defaults": {
      "model": {
        "primary": "cheaper-inference/gpt-5.4"
      }
    }
  },
  "models": {
    "providers": {
      "cheaper-inference": {
        "baseUrl": "https://api.cheaperinference.com/v1",
        "apiKey": "${CHEAPER_INFERENCE_API_KEY}",
        "api": "openai-completions",
        "models": [
          {
            "id": "gpt-5.4",
            "name": "GPT-5.4 via Cheaper Inference"
          }
        ]
      }
    }
  }
}
```

```
}  
  ]  
}  
}  
}  
}
```

Important: keep `api` set to `openai-completions`. Add another object to `models` for each Cheaper Inference model ID you want OpenClaw to expose.

Capability metadata: OpenClaw supplies generic context, output-token, reasoning, and input-modality defaults when optional model fields are omitted. Add explicit values only after verifying them for the exact model you selected.

Hermes Agent

[Official provider docs](#)

Use Hermes' custom endpoint option.

1. Run `hermes model`.
2. Choose **Custom endpoint (self-hosted / VLLM / etc.)**.
3. Enter the base URL above, your API key, and an exact model ID such as `gpt-5.4`.
4. When Hermes asks for the API mode, choose **Chat Completions**.

For a manual setup, put `CHEAPER_INFERENCE_API_KEY=ir_live_YOUR_API_KEY` in `~/.hermes/.env`, then edit `~/.hermes/config.yaml`:

HERMES AGENT CONFIGURATION

Copy

```
custom_providers:
  - name: cheaper-inference
    base_url: https://api.cheaperinference.com/v1
    key_env: CHEAPER_INFERENCE_API_KEY
    api_mode: chat_completions

model:
  default: gpt-5.4
  provider: custom:cheaper-inference
```

Recommended: use `hermes model` so Hermes writes the configuration in its current format. Protect `~/.hermes/.env` because it contains your key.

OpenCode

[Official provider docs](#)

Configure an OpenAI-compatible custom provider.

1. Open `/connect`, choose **Other**, enter `cheaper-inference`, and add your API key.
2. Add the provider below to `opencode.json`, then use `/models` to select it.

OPENCODE CONFIGURATION

Copy

```
{
  "$schema": "https://opencode.ai/config.json",
  "provider": {
    "cheaper-inference": {
```

```

    "npm": "@ai-sdk/openai-compatible",
    "name": "Cheaper Inference",
    "options": {
      "baseUrl": "https://api.cheaperinference.com/v1"
    },
    "models": {
      "gpt-5.4": {
        "name": "GPT-5.4"
      }
    }
  }
}
}
}

```

Important: use `@ai-sdk/openai-compatible` so OpenCode sends Chat Completions requests. Add more entries under `models` using exact IDs from the live catalog.

OpenWork

[Official OpenWork project](#)

Use Cheaper Inference in the OpenCode-powered desktop agent.

The OpenCode schema is intentional: OpenWork uses OpenCode as its model engine and reads OpenCode provider configuration for each workspace.

1. Open a workspace in OpenWork and create `opencode.json` in that workspace's root folder.
2. Add the provider configuration below, then reopen or reload the workspace.
3. Open **Settings** → **AI Providers** → **Connect provider**, choose **Cheaper Inference**, and paste your API key.
4. Select a Cheaper Inference model from OpenWork's model picker and start a task.

OPENWORK CONFIGURATION

Copy

```

{
  "$schema": "https://opencode.ai/config.json",
  "provider": {
    "cheaper-inference": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "Cheaper Inference",

```

```
"env": ["CHEAPER_INFERENCE_API_KEY"],
"options": {
  "baseUrl": "https://api.cheaperinference.com/v1"
},
"models": {
  "gpt-5.6-sol": {
    "name": "GPT-5.6 Sol"
  },
  "gpt-5.6-terra": {
    "name": "GPT-5.6 Terra"
  },
  "deepseek-v4-flash": {
    "name": "DeepSeek V4 Flash"
  }
}
}
```

Keep the key out of the file: the `env` declaration tells OpenWork that this provider accepts an API key; it does not contain the secret. OpenWork stores the key locally through OpenCode after you connect the provider. Add or replace model entries using exact IDs from the [live catalog](#).

If Cheaper Inference does not appear: completely reload the workspace engine after saving the file. If it is still missing, capture the exact error and your OpenWork version before contacting support.



**Более дешевый
вывод**
КОМПАНИЯ KEAK

Документация по
API

История изменения
цен

Открытые API-
ключи

1. Open **Admin Settings** → **Connections** → **OpenAI**.
2. Click **Add Connection**.
3. Enter `https://api.cheaperinference.com/v1` and your API key, then save.
4. Select a discovered model. If needed, add exact IDs under **Model IDs (Filter)**.

Feature scope: chat and model discovery work through the Cheaper Inference connection. Configure a separate provider for Open WebUI features that require embeddings, speech-to-text, or text-to-speech.

Cline

[Official provider docs](#)

Use the OpenAI Compatible provider in VS Code.

1. Open Cline settings and choose **OpenAI Compatible** as the API provider.
2. Set the Base URL to `https://api.cheaperinference.com/v1`.
3. Paste your API key and enter an exact model ID such as `gpt-5.4`.
4. Save, then start a small task and confirm the request appears in Cheaper Inference History.

Advanced model settings: enable image support, computer/tool use, and context or output limits only when the selected model supports them. These settings are model-specific, even when the same OpenAI-compatible connection is reused.

Cursor

[Official API key docs](#)

Use Cursor's OpenAI base-URL override for compatible models.

Supported with limitations: Cursor can send Ask and Agent requests for OpenAI-family models through Cheaper Inference. The gateway bridges Cursor's streamed `ApplyPatch` custom tool and older Responses-shaped BYOK requests. Cursor controls the agent runtime and some specialized features separately, so this is not a replacement for every Cursor-hosted model or feature.

1. Update Cursor to the latest stable version.
2. [Create a dedicated Cheaper Inference key](#) with **Inference** scope. Limit it to the models Cursor should use, then add appropriate request and monthly-spend limits. Fund the wallet before verification if the workspace has no available balance.
3. Open **Cursor Settings** → **Models**, then find **OpenAI API Key**.
4. Paste the dedicated key and enable **Override OpenAI Base URL**.
5. Set the base URL to `https://api.cheaperinference.com/v1`, then click **Verify**.
6. Select an OpenAI-family model whose exact ID appears in the [live model catalog](#).

7. Test a short prompt in **Ask** mode, then ask **Agent** mode to inspect a file and make one harmless edit. Confirm each request appears in Cheaper Inference History with

Global override: Cursor applies one OpenAI base URL and key across its OpenAI-family models; it cannot keep separate provider settings per custom model. Turn off the override when you want those models to use Cursor's normal routing again.

Key security: Cursor assembles requests through its service, so the gateway sees Cursor's outbound address rather than your workstation's address. Do not restrict the key to your workstation IP. If your organization has verified Cursor egress ranges, allow those ranges; otherwise rely on the dedicated key's model, rate, spend, expiry, and scope controls.

Model and feature scope: use the exact model ID Cursor forwards. If an ID already exists in Cursor, select that built-in entry instead of adding a duplicate. Claude- and Gemini-family IDs do not use Cursor's OpenAI override. Tab completion and other features that require specialized Cursor models continue to use Cursor's own infrastructure.

Compatibility path: Cursor sends these requests to `/v1/chat/completions`. Cheaper Inference accepts ordinary Ask requests, translates Agent's streamed `ApplyPatch` tool calls, and converts the older Responses-shaped BYOK format without changing the response shape Cursor expects.

► Troubleshooting

Aider

[Official compatible API docs](#)

Point the terminal pair programmer at the compatible endpoint.

AIDER CONFIGURATION

Copy

```
export OPENAI_API_BASE=https://api.cheaperinference.com/v1
export OPENAI_API_KEY=ir_live_YOUR_API_KEY
```

```
cd /path/to/your/project
aider --model openai/gpt-5.4
```

Windows PowerShell:

AIDER CONFIGURATION

Copy

```
$env:OPENAI_API_BASE = "https://api.cheaperinference.com/v1"
$env:OPENAI_API_KEY = "ir_live_YOUR_API_KEY"

Set-Location C:\path\to\your\project
aider --model openai/gpt-5.4
```

Model naming: keep Aider's `openai/` prefix, followed by the exact Cheaper Inference model ID. If Aider warns that a newer model is unknown, its official model settings can be used to describe the model's context and edit format.

LibreChat

[Official custom endpoint docs](#)

Add a custom OpenAI-compatible endpoint for a team chat deployment.

Add the key to LibreChat's `.env` :

LIBRECHAT CONFIGURATION

Copy

```
CHEAPER_INFERENCE_API_KEY=ir_live_YOUR_API_KEY
```

Add the endpoint to `librechat.yaml` , then restart LibreChat:

LIBRECHAT CONFIGURATION

Copy

```
version: 1.3.13
endpoints:
  custom:
    - name: Cheaper Inference
      apiKey: '${CHEAPER_INFERENCE_API_KEY}'
      baseURL: 'https://api.cheaperinference.com/v1'
      models:
        default:
          - gpt-5.4
          - claude-opus-4.6
```

```
fetch: true
titleConvo: true
titleModel: gpt-5.4
modelDisplayLabel: Cheaper Inference
```

Claude model IDs: keep this as a custom OpenAI-compatible endpoint even when selecting a Claude model. Cheaper Inference translates the common request format to the serving provider.

Continue

[Official OpenAI provider docs](#)

Add a model to Continue's YAML configuration.

CONTINUE CONFIGURATION

Copy

```
name: Cheaper Inference
version: 0.0.1
schema: v1

models:
  - name: GPT-5.4 through Cheaper Inference
    provider: openai
    model: gpt-5.4
    apiBase: https://api.cheaperinference.com/v1
    apiKey: ir_live_YOUR_API_KEY
    useResponsesApi: false
```

Recommended for Continue: keep `useResponsesApi: false` so Continue uses the broadly compatible Chat Completions path. The Responses endpoint is currently targeted at stateless Codex workloads.

Verify the connection

1. Start with a short prompt and a model shown in the [live catalog](#).
2. Confirm the response completes in the app.

3. Open your Cheaper Inference dashboard and verify the model, token usage, charge, `code /v1/responses` , and Claude Code can use `/v1/messages` . For other apps, use OpenAI-compatible **Chat Completions** unless the guide says otherwise. Configure a separate provider for embeddings, audio, stored Responses conversations, or hosted web and file search. For raw HTTP examples, parameters, and error handling, see the [API documentation](#).

A Keak company

keak.com

Privacy policy

Terms

DPA

Trust Center

Status